ECOLE NATIONALE SUPERIEURE DES TECHNOLOGIES AVANCEES
ENSTA ALGER · THIRD YEAR ENGINEERING , AI & SS

PROJECT REPORT

Cryptography Applied to Electronic Voting

RSA · Blind Signatures · Hash Functions · Digital Signatures

PROJECT TEAM	
1	**DAHMOUN Mouaine Aymen**
2	**BENCHENINA Shouhaib Meheni**
3	**BENZINA Mohamed**
4	**MEZAACHE Abdessamie**
5	**FANTAZI Sami Ilyes**

Supervised by: Mrs. Souad KHERROUBI

Module: Applied Cryptography | Academic Year 2025–2026
Due: Last week of May 2026

TABLE OF CONTENTS

1. INTRODUCTION

Electronic voting (e-voting) represents a significant evolution in democratic participation, offering improved convenience, accessibility, and speed compared to traditional paper-based systems. Citizens can cast ballots from home over an extended period, and results can be tallied almost instantaneously. However, the digitisation of the voting process introduces new and severe security concerns.

The fundamental challenge is that electronic ballots can be duplicated, altered, or fabricated on a large scale without physical evidence. A robust cryptographic solution must simultaneously guarantee several requirements that can appear conflicting:

- Authenticity: Only registered, eligible citizens may vote.
- Uniqueness: Each eligible citizen may vote exactly once.
- Anonymity: No entity can link a cast ballot to its voter.
- Integrity: No ballot can be altered once submitted.
- Verifiability: Each voter can confirm their vote was counted correctly.
- Security: No participant , internal or external , can commit fraud.

This project, assigned by Mrs. Souad KHERROUBI at ENSTA Alger (Third Year Engineering, AI & SS), develops a complete solution based on a blind-signature electronic voting protocol. The document presents the theoretical cryptographic background, solutions to all four prescribed exercises, a UML protocol diagram, a security analysis, and a full description of our Python implementation.

The protocol draws on four cryptographic primitives: the RSA public-key cryptosystem, digital signatures, cryptographic hash functions, and blind signatures. Each plays a distinct and essential role in achieving the security properties listed above, as developed in the sections that follow.

> **Project scope:** This report covers all deliverables specified in the project brief: exercise solutions (Exercises 1–4), a Python implementation using the Cryptography library, a web-deployed application, a UML sequence diagram, a security analysis, and this report. Live deployment: https://cryptovote-obm3.onrender.com/

2. THEORETICAL BACKGROUND

2.1 The RSA Cryptosystem

RSA (Rivest–Shamir–Adleman, 1978) is the foundational asymmetric cryptosystem used throughout this protocol. Its security relies on the computational difficulty of factoring the product of two large prime numbers , a problem for which no efficient classical algorithm is known.

Key Generation

1. Choose two distinct large primes p and q.
2. Compute the modulus N = p × q and the Euler totient φ(N) = (p−1)(q−1).
3. Choose public exponent e such that 1 < e < φ(N) and gcd(e, φ(N)) = 1.
4. Compute private exponent d such that e·d ≡ 1 (mod φ(N)) via the Extended Euclidean Algorithm.
5. Public key: (e, N). Private key: (d, N).

Encryption and Decryption

Encrypt: $C = M^e \pmod N$	Decrypt: $M = C^d \pmod N$

Correctness follows from Euler's theorem: $M^{ed} \equiv M \pmod N$ when gcd(M, N) = 1, since $e \cdot d \equiv 1 \pmod{\varphi(N)}$ implies $M^{ed} = M^{\{1+k\varphi(N)\}} \equiv M \pmod N$.

Digital Signatures with RSA

RSA signatures reverse the key roles: the signer uses the private key, and anyone with the public key can verify. This provides authentication and non-repudiation.

Sign: $S = M^d \pmod N$	Verify: $S^e \equiv M \pmod N$

2.2 Digital Signatures

A digital signature scheme provides authentication and non-repudiation. In the voting protocol, the administrator issues a digital signature on each ballot, certifying its legitimacy without learning its content (thanks to blind signatures). The counter verifies these signatures during tallying to reject any forged or tampered ballot.

The key properties required are: correctness (valid signatures always verify), unforgeability (only the private key holder can produce valid signatures), and non-repudiation (a signer cannot deny having signed).

2.3 Hash Functions and Preimage Resistance

A cryptographic hash function $H: \{0,1\}^* \to \{0,1\}^n$ maps an arbitrary-length input to a fixed-size digest. Three properties are required for cryptographic use:

- Preimage resistance (one-wayness): Given $H(x)$, it is computationally infeasible to find any x.
- Second-preimage resistance: Given x, it is infeasible to find $x' \neq x$ such that $H(x') = H(x)$.
- Collision resistance: It is infeasible to find any two distinct x, x' with $H(x) = H(x')$.

In this protocol, only the TTH fingerprints of N2 codes are stored by the commissioner. Preimage resistance ensures that the commissioner cannot reverse-engineer valid N2 codes from the stored fingerprints, preventing the fabrication of fraudulent ballots.

The TTH (Toy Tetragraph Hash) is an educational hash function used in this project. It encodes alphanumeric characters (letters as A=0…Z=25, digits as-is), divides the input into 4-character blocks, and applies iterative modular mixing to produce a compact 4-character digest. While not cryptographically secure for production use, it clearly illustrates the concept of one-way fingerprinting.

2.4 Blind Signatures

A blind signature scheme, introduced by David Chaum (1983), allows a signer to sign a message without seeing its content. The analogy is a document placed inside a carbon-paper envelope: the signature goes through the envelope onto the document without the signer reading it.

In the voting context, this is critical: the administrator must authenticate each ballot (ensuring it is legitimate) without learning the voter's choice. The RSA blind signature protocol proceeds in three steps:

6. **Blinding:** The voter picks random k with $\gcd(k, N) = 1$ and computes $m' = m \cdot k^e \pmod N$, then sends m'.
7. **Signing:** The administrator applies their private key and returns $m'' = (m')^d \pmod N$.
8. **Unblinding:** The voter computes $s = m'' \cdot k^{-1} \pmod N$, obtaining a valid RSA signature s on the original message m.

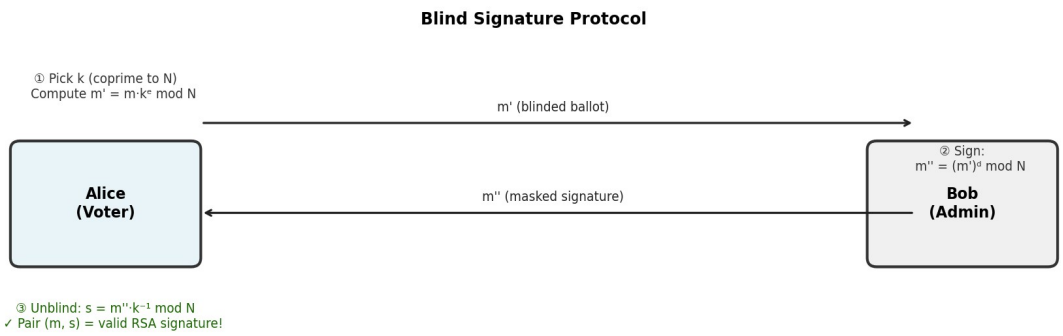

Figure 1 , Blind Signature Protocol: Alice (Voter) and Bob (Administrator)

3. SYSTEM ARCHITECTURE

3.1 Protocol Entities

The protocol distributes trust across four independent server entities. No single entity has sufficient knowledge to commit undetected fraud; each is constrained by what information it can access and what cryptographic operations it performs:

Entity	Cryptographic Keys	Role & Constraints
Commissioner	None	Holds list of valid N1 codes and TTH(N2) fingerprints. Never contacts votes directly. Cannot reconstruct N2 from hashes.
Administrator	RSA Public/Private	Blind-signs ballots. Sees masked messages only. Cannot link signed ballot to voter or reconstruct N2. Stops signing after voting closes.
Anonymizer	None	Receives counter-encrypted ballots. Verifies N1, strikes it from list, records encrypted vote in ballot box.
Counter	RSA Public/Private	Decrypts ballots after voting closes, verifies signatures and N2 fingerprints, tallies and publishes results.

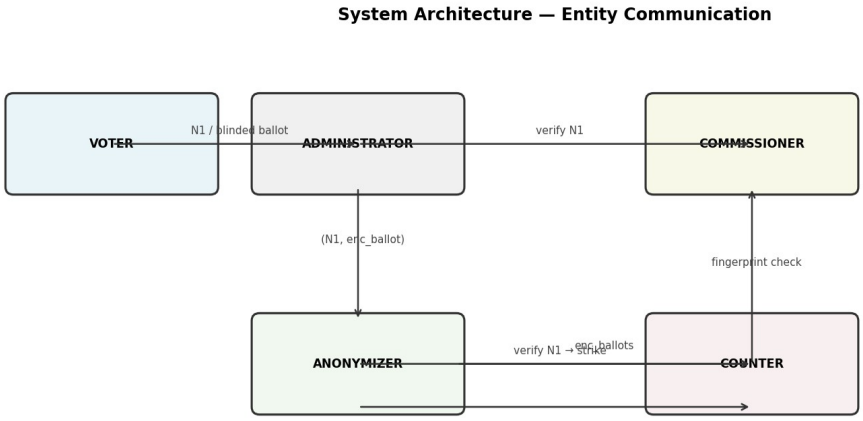

Figure 2 , System Architecture: Entity communication during the protocol

3.2 Voter Code Scheme

Before the election, each voter is issued a physical card containing two distinct 12-character alphanumeric codes:

- N1 (eligibility code): Proves the right to vote. The commissioner maintains the complete list of valid N1 codes. N1 is presented to the administrator, then to the anonymizer, where it is permanently struck from the list upon use, preventing double-voting.
- N2 (anonymity code): Embedded in the ballot to allow the voter to verify their vote post-election. Only TTH(N2) is stored by the commissioner; the original N2 list is destroyed. This ensures the commissioner cannot fabricate valid ballots.

Code space analysis: With 12 alphanumeric characters (10 digits + 26 letters = 36 symbols), the total number of distinct codes is 36^12 ≈ 4.7 × 10^18. For a population of one million voters, the probability of randomly guessing a valid N1 code is negligible:

$$P(\text{random N1 valid}) = 10^6 / 36^{12} \approx 2.1 \times 10^{-13} \quad \text{(negligible)}$$

Random bits are also appended to each ballot when it is formed. These nonce bits prevent the anonymizer from correlating the encrypted ballots it receives with the (N2, vote) pairs published after the election, since identical votes produce different ciphertexts.

4. COMPLETE VOTING PROTOCOL

4.1 Election Preparation

The preparation phase establishes the cryptographic infrastructure before voting begins. The following steps are carried out:

9. Key generation: The administrator generates RSA key pair (e_a, N_a) / (d_a, N_a). The counter generates RSA key pair (e^c, N^c) / (d^c, N^c). Both public keys are published to all participants.
10. Voter card distribution: Each registered voter receives a physical card with two unique 12-character alphanumeric codes: N1 (eligibility code) and N2 (anonymity code).
11. Commissioner setup: The commissioner receives the complete list of valid N1 codes. All N2 codes are hashed via TTH, and only the fingerprint list TTH(N2) is given to the commissioner. The original N2 list is then permanently destroyed.
12. System validation: All four servers (commissioner, administrator, anonymizer, counter) are online, and secure encrypted channels between them are established.

4.2 Voting Phase

Once the election begins, each voter follows these steps using the official voting software:

13. Eligibility verification: The voter submits N1 to the administrator, who forwards it to the commissioner for validation. If valid, the right to vote is granted.
14. Ballot creation: The voter selects their vote choice and enters N2. The software constructs a ballot as: ballot = (vote, N2, random_bits).
15. Blinding: The voter generates a random blinding factor k (coprime to N_a) and computes the blinded ballot: $m' = \text{ballot} \cdot k^{e_a} \pmod{N_a}$.
16. Blind signing: The blinded ballot is sent to the administrator, who applies their private key: $m'' = (m')^{d_a} \pmod{N_a}$, then returns m'' to the voter. The administrator does not know the original ballot.
17. Unblinding: The voter computes the signature: $s = m'' \cdot k^{-1} \pmod{N_a}$. The pair (ballot, s) is a valid RSA signature by the administrator.
18. Submission: The voter encrypts the signed ballot using the counter's public key: $\text{enc\_ballot} = (\text{ballot}, s)^{e^c} \pmod{N^c}$. The voting software sends (N1, enc_ballot) to the anonymizer.
19. Anonymizer recording: The anonymizer verifies N1 with the commissioner, who strikes N1 from the valid list (preventing double-voting). The anonymizer records enc_ballot in the ballot box.

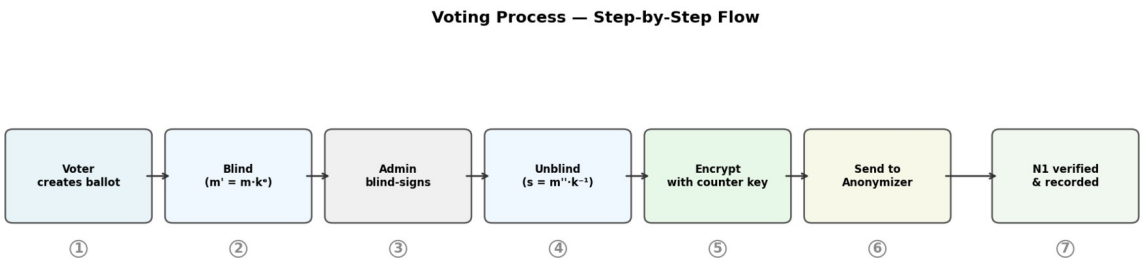

Figure 3 , Voting Process: Step-by-step flow from Voter to Anonymizer

4.3 Counting Phase

Once the voting period closes, the counter tallies all ballots:

20. Decryption: The counter receives all enc_ballots from the anonymizer and decrypts each using its private key: (ballot, s) = enc_ballot^{d_c} (mod N_c).
21. Signature verification: For each ballot, the counter verifies the administrator's signature: s^{e_a} ≡ ballot (mod N_a). Any ballot failing verification is rejected.
22. N2 fingerprint check: The counter extracts N2 from each ballot and sends it to the commissioner, who verifies that TTH(N2) appears in the valid fingerprint list.
23. Vote counting: Ballots passing both checks are counted. The aggregate results are computed.
24. Publication: The pairs (N2, vote) are published, allowing any voter to verify that their vote was recorded by looking up their personal N2 code in the public list.

5. EXERCISE SOLUTIONS

5.1 Exercise 1 , Proof of Blind Signature Validity

> **Statement:** Prove that s = m″/k (mod N) is a valid RSA signature of message m by Bob. Apply the RSA signature verification algorithm to the pair (m, s).

Setup Recap

Bob's RSA system: public key (e, N), private key d, with e·d ≡ 1 (mod φ(N)).

Alice sends blinded message $m' = m \cdot k^e \pmod N$. Bob returns $m'' = (m')^d \pmod N$. Alice computes $s = m'' \cdot k^{-1} \pmod N$.

Formal Proof

To verify that (m, s) is a valid RSA signature by Bob, we must show that $s^e \equiv m \pmod N$.

Step 1: Start from the definition of s.

$$s = m'' \cdot k^{-1} \pmod N$$

Step 2: Substitute $m'' = (m')^d$.

$$s = (m')^d \cdot k^{-1} \pmod N$$

Step 3: Substitute $m' = m \cdot k^e$.

$$s = (m \cdot k^e)^d \cdot k^{-1} \pmod N = m^d \cdot k^{ed} \cdot k^{-1} \pmod N$$

Step 4: Apply RSA correctness. Since e·d ≡ 1 (mod φ(N)), Euler's theorem gives k^{ed} ≡ k^1 ≡ k (mod N).

$$s = m^d \cdot k \cdot k^{-1} \pmod N = m^d \pmod N$$

Step 5: Raise both sides to the power e.

$$s^e = (m^d)^e = m^{de} = m \pmod N$$

> **Conclusion:** Since $s^e \equiv m \pmod N$, the pair (m, s) satisfies the RSA signature verification condition. Therefore s is a valid RSA signature of m by Bob, even though Bob never saw the original message m. □

5.2 Exercise 2 , RSA Key Finding & Blind Signature Demo

> **Statement:** Given RSA modulus N = 55 and public key e = 27. (1) Find private key d. (2) Simulate the blind signature protocol with a friend.

5.2.1 Finding the Private Key d

Step 1: Factorise N = 55.

$$N = 55 = 5 \times 11 \quad \rightarrow \quad p = 5,\ q = 11$$

Step 2: Compute Euler's totient.

$$\varphi(N) = (p-1)(q-1) = 4 \times 10 = 40$$

Step 3: Verify gcd(e, φ(N)) = gcd(27, 40). Using the Euclidean algorithm: 40 = 1×27+13; 27 = 2×13+1. So gcd(27,40) = 1 ✓ (e is valid).

Step 4: Find d such that e·d ≡ 1 (mod φ(N)), i.e., 27d ≡ 1 (mod 40). Testing d = 3:

$$27 \times 3 = 81 = 2 \times 40 + 1 \equiv 1 \pmod{40} \quad \checkmark$$

Result: Private key d = 3. Full key pair: public (e=27, N=55), private (d=3, N=55).

5.2.2 Blind Signature Protocol Simulation

We choose message m = 7 (a grade to be signed) and masking factor k = 3. We verify: gcd(3, 55) = 1 since 55 = 5×11 and 3 is coprime with both. The full simulation:

	Actor	Formula / Action	Value
	Alice , Blind the message	$m' = m \cdot k^e \bmod N = 7 \cdot 3^{27} \bmod 55$	**m' = 19**
	Alice , Send m′ to Bob	Bob receives blinded message (cannot determine m = 7 without knowing k)	✉ m′ = 19
	Bob , Apply private key	$m'' = (m')^d \bmod N = 19^3 \bmod 55$	**m'' = 39**
	Bob , Return m″	Alice receives masked signature	✉ m'' = 39
	Alice , Compute k^{-1} mod N	3 × 37 = 111 = 2×55+1 ≡ 1 mod 55 ✓	**$k^{-1} = 37$**
	Alice , Unblind to get s	$s = m'' \cdot k^{-1} \bmod N = 39 \times 37 \bmod 55$	**s = 13**
	Verification , Anyone	$s^e \bmod N = 13^{27} \bmod 55$ should equal m = 7	**$13^{27} \bmod 55 = 7$ ✓**

Result: Alice holds (m=7, s=13): a valid RSA signature by Bob on m=7. Bob signed it without ever seeing m=7, only the blinded value m′=19.

5.3 Exercise 3 , Hash Functions and the TTH

> **Statement:** (1) Which hash property prevents reconstructing N2 from its fingerprint? (2) Choose personal N1, N2 codes of 12 characters. (3) Compute TTH(N2).

5.3.1 Relevant Hash Property

The property that prevents reconstructing a valid N2 code from its TTH fingerprint is preimage resistance (one-wayness).

Definition: A hash function H is preimage-resistant if, given a hash value h, it is computationally infeasible to find any input x such that H(x) = h. In other words, H cannot be efficiently inverted.

In the protocol, the commissioner stores TTH(N2) for each voter. Even knowing h = TTH(N2), preimage resistance guarantees that the commissioner cannot recover N2 from h. Without the actual N2 value, they cannot construct a valid ballot (which requires N2), preventing ballot stuffing.

A secondary property, collision resistance, ensures that no adversary can find a different code N2′ ≠ N2 with TTH(N2′) = TTH(N2), preventing code substitution attacks where a fraudulent ballot uses a different N2 that happens to hash to the same fingerprint.

5.3.2 Personal Codes and TTH Computation

Code	Value (12 alphanumeric chars)	Hash / Note
N1	PU53CXPWV1PB	Eligibility code (not hashed)
N2	BC23JK479MNP	**TTH(N2) = SQTV**

TTH Computation Trace for N2 = BC23JK479MNP

Encoding rule: letters → alphabet index (A=0…Z=25), digits kept as-is.

```
B=1, C=2, 2, 3  |  J=9, K=10, 4, 7  |  9, M=12, N=13, P=15
```

The input is split into three 4-character blocks. Initial state s = [0,0,0,0]. For each block, the state is updated iteratively: s[i] = (s[i] + block[i]) mod 26 for each position, then mixed with s[i] = (s[i] + s[(i+1)%4]) mod 26.

```
            After block 1 [1,2,2,3]:   s = [3,4,5,4]

After block 2 [9,10,4,7]:  s = [16,18,14,18]  → further mix →
                        [16,18,14,18]

   Final state:  [18,16,19,21]  →  S=18, Q=16, T=19, V=21

                TTH("BC23JK479MNP")  =  SQTV
```

5.4 Exercise 4 , Anonymous Course Evaluation

> **Statement:** Simulate an anonymous vote evaluating the Asymmetric Cryptography course. Each student votes a score out of 10. Counter RSA public key: (e=3, N=583).

5.4.1 Protocol Execution

Working in groups, each student acts as a voter. One member acts as commissioner (receives N1 list and TTH(N2) fingerprints), one as administrator (uses keys from Exercise 2), and the teacher acts as anonymizer. The steps:

25. Distribute N1 codes and TTH(N2) fingerprints to the commissioner.
26. Create ballot = (grade, N2), e.g., (8, BC23JK479MNP).
27. Blind-sign the ballot using the administrator's keys from Exercise 2 (N=55, e=27, d=3).
28. Encrypt the grade using the counter's RSA public key (e=3, N=583).
29. Send the encrypted, signed vote to the anonymizer (teacher) for recording.

5.4.2 Counter Key Analysis (N=583, e=3)

Factorising the counter's modulus:

$$583 = 11 \times 53 \quad \rightarrow \quad p = 11,\ q = 53$$

$$\varphi(583) = (11-1)(53-1) = 10 \times 52 = 520$$

Finding private key d: solve $3d \equiv 1 \pmod{520}$. Applying the Extended Euclidean Algorithm:

$$3 \times 347 = 1041 = 2 \times 520 + 1 \equiv 1 \pmod{520} \quad \checkmark \quad \rightarrow \quad d = 347$$

Operation	Formula	Result
Encrypt vote = 8	$8^3 \bmod 583$	**512**
Decrypt (counter uses d=347)	$512^{347} \bmod 583$	**8** ✓
Round-trip verified	Decrypted value = original vote	**RSA correct** ✓

6. UML SEQUENCE DIAGRAM

The following UML sequence diagram provides a complete, formal representation of the electronic voting protocol. It shows all five participants (Voter, Administrator, Commissioner, Anonymizer, Counter) and all message exchanges across the three phases: election preparation, voting, and counting.

Standard UML notation is used: lifelines (dashed vertical lines), activation boxes (processing periods), synchronous request messages (solid arrows), and return messages (dashed arrows). Shaded banners mark the three protocol phases.

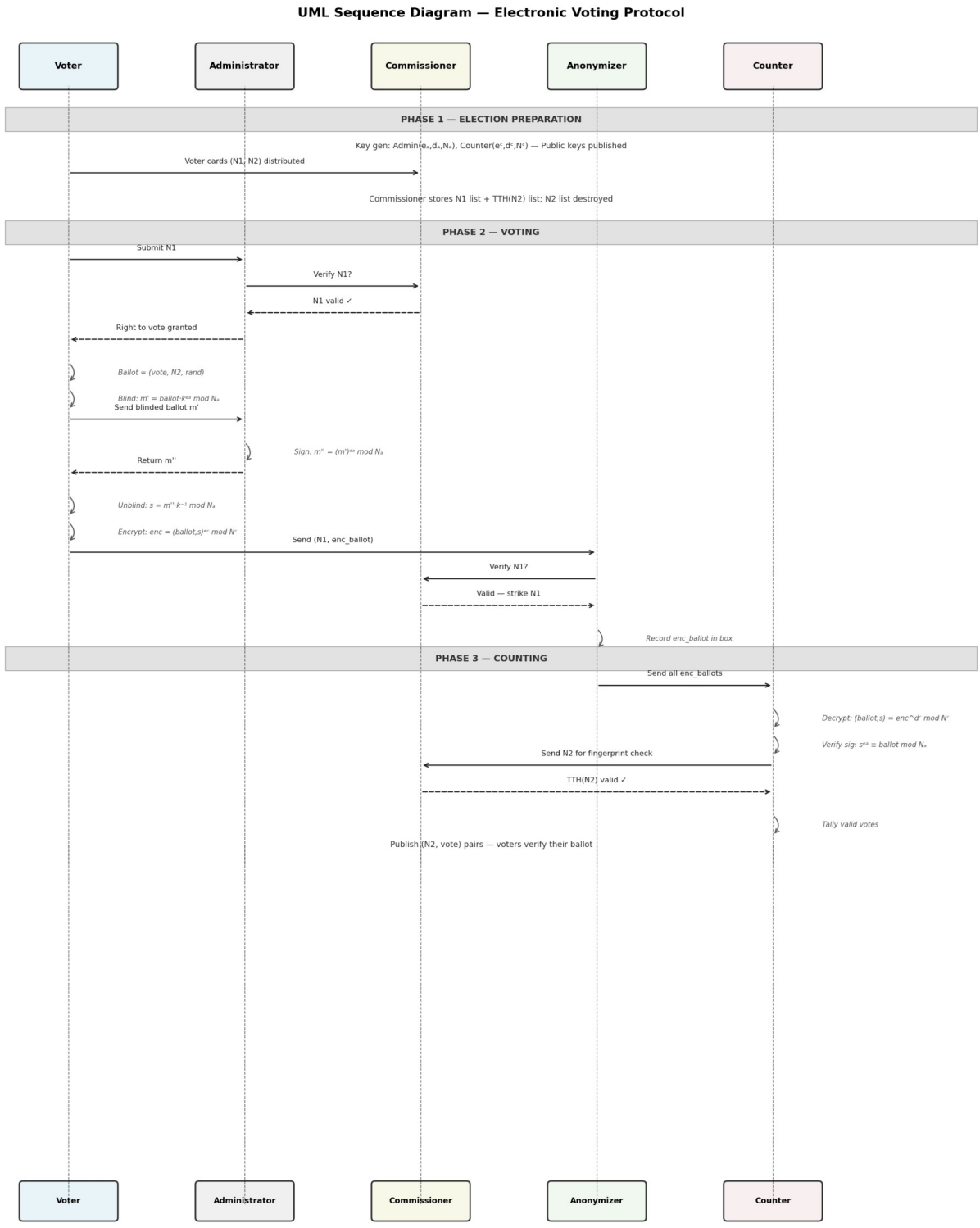

Figure 4 , UML Sequence Diagram: Complete Electronic Voting Protocol

Diagram Notation Key

Symbol	Meaning
——————▶	Synchronous message (request / data transmission)
- - - - - ▶	Return message (response / acknowledgement)

[PHASE BANNER]	Protocol phase separator (shaded grey background)
Self-loop arrow	Internal computation (e.g., blinding, hashing, signing)

7. PROTOCOL SECURITY ANALYSIS

7.1 External Security

The protocol is secure against external attackers in the following senses:

- Without N1: No external party can claim the right to vote. The probability of guessing a valid N1 is approximately 2×10^{-13} for a population of 10^6, making brute-force attacks computationally infeasible.
- Without N2: No valid ballot can be submitted, since the counter verifies TTH(N2) during tallying. Furthermore, the administrator stops signing new ballots after the voting period closes.
- Eavesdropping: The ballot is encrypted with the counter's public key, so network eavesdroppers cannot read its content. Signed ballots are submitted anonymously through the anonymizer.

7.2 Internal Entity Security

The following table analyses the security constraints on each internal entity and how the protocol prevents fraud:

Entity	Potential Threat	Cryptographic Mitigation
Commissioner	Accept/reject votes fraudulently; stuff ballot box	Has no contact with actual votes or ballots. Stores only TTH(N2): preimage resistance prevents N2 reconstruction, so valid ballots cannot be fabricated.
Administrator	Learn vote content; link vote to voter; forge ballots	Blind signatures ensure the administrator never sees ballot content. Has no access to N2 codes. Once voting closes, signing stops: no more valid ballots can be created.
Anonymizer	Read ballot content; correlate submissions with published pairs	All ballots are encrypted with the counter's public key (unreadable). Random nonce bits ensure identical votes produce different ciphertexts, breaking any correlation with published (N2, vote) pairs.
Counter	Modify vote counts; alter individual ballots; forge new votes	Cannot forge new valid signatures (administrator has stopped signing). Published (N2, vote) pairs allow full public audit. Any manipulation is detectable.

7.3 Security Properties Summary

Property	How Achieved in This Protocol
Eligibility	N1 codes verified by commissioner; struck from valid list after use, preventing double-voting.
Anonymity	Blind signatures prevent administrator from linking ballot to voter; random nonce bits prevent correlation by anonymizer.
Integrity	RSA digital signature on each ballot verified by counter using administrator's public key; any tampered ballot is rejected.
Confidentiality	Ballots encrypted with counter's public key; only counter can decrypt. Anonymizer sees only encrypted envelopes.
Verifiability	Post-election publication of (N2, vote) pairs; each voter can look up their N2 code to confirm their vote was correctly counted.
Non-repudiation	Administrator's digital signature proves ballot authenticity; counter cannot fabricate new valid ballots after voting closes.
Distributed trust	No single entity holds all information required to commit fraud; the four-entity architecture ensures attacks require multi-party collusion.

8. PYTHON IMPLEMENTATION

8.1 Application Architecture

Our Python implementation fully realises the electronic voting protocol described in this report. It is built using the Python Cryptography library (as specified in the project requirements) and exposed as a web application using Flask. The live deployment is publicly accessible at: https://cryptovote-obm3.onrender.com/

The application is structured as independent modules, each corresponding to one protocol entity or utility function:

Module / File	Responsibility
rsa_utils.py	RSA key generation using the Cryptography library, modular exponentiation via pow(base, exp, mod), encryption, decryption, signature creation and verification.
blind_signature.py	Blinding factor generation (random k coprime to N via os.urandom), message blinding, blind signature verification, and unblinding using modular inverse.
tth_hash.py	Toy Tetragraph Hash: alphanumeric encoding, 4-character block splitting, iterative modular mixing, and digest output as a 4-character string.
commissioner.py	N1 code list management, TTH(N2) fingerprint storage, eligibility verification service, and N1 strike-off (preventing double-voting).
administrator.py	Blind signing service using RSA private key; state-machine enforcement that refuses new signatures after the election closes.
anonymizer.py	Ballot box management: receives (N1, enc_ballot) pairs, verifies N1 with commissioner, records encrypted votes in an ordered list.
counter.py	RSA ballot decryption using private key, administrator signature verification, N2 fingerprint forwarding to commissioner, and vote tallying.
app.py	Flask REST API and web interface: routes for voter portal, admin dashboard (password-protected), results publication, and protocol visualisation.

8.2 Application Screenshots

Home Page , Role Selection

The application's home page presents two entry points: the Voter Portal for casting anonymous ballots, and the Admin Panel (password-protected) for managing the entire election lifecycle from setup through to results publication.

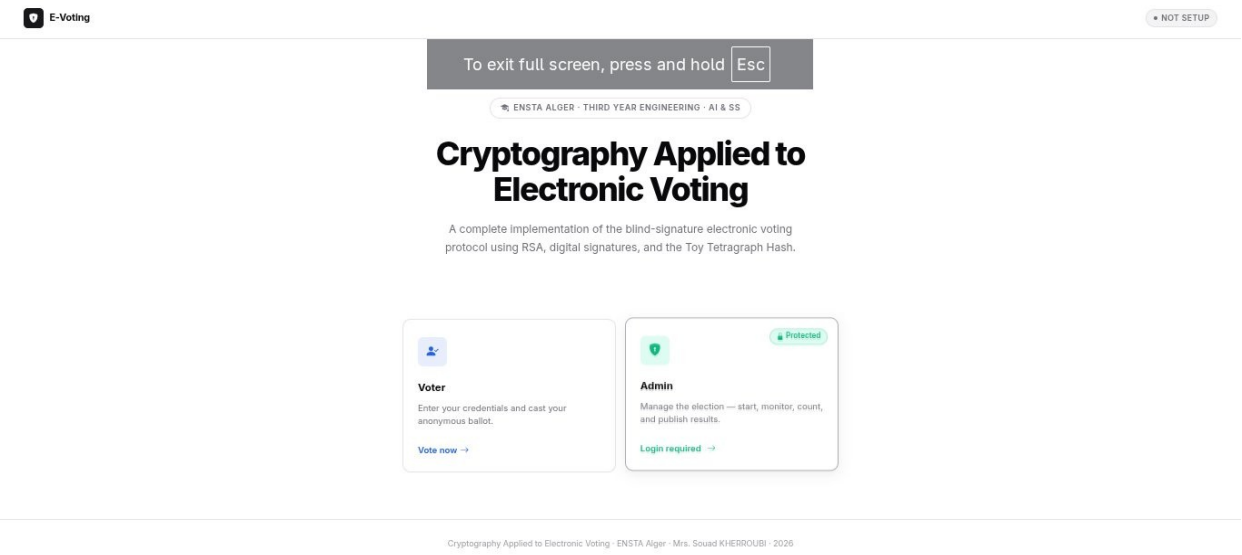

Figure 5 , Application Home Page: Voter and Admin entry points at cryptovote-obm3.onrender.com

Voter Portal , Live Protocol Execution

The voter portal provides a step-by-step visual execution of the cryptographic protocol. Each phase is displayed in sequence with the actual mathematical computations performed in real time. The example shown uses N1 = PU53CXPWV1PB and a grade vote of 10:

- Step 1 , Eligibility Check: N1 is validated and confirmed as valid and unused. Right to vote is granted.
- Step 2 , Ballot Creation & Blinding: The grade m=10 is blinded with factor k=29, producing $m' = 10 \cdot 29^{27} \bmod 55 = 5$. The blinded ballot is sent to the administrator.
- Step 3 , Administrator Blind-Signs: Admin applies d=3 to blinded ballot: $m'' = 5^3 \bmod 55 = 15$. The masked signature is returned to the voter.

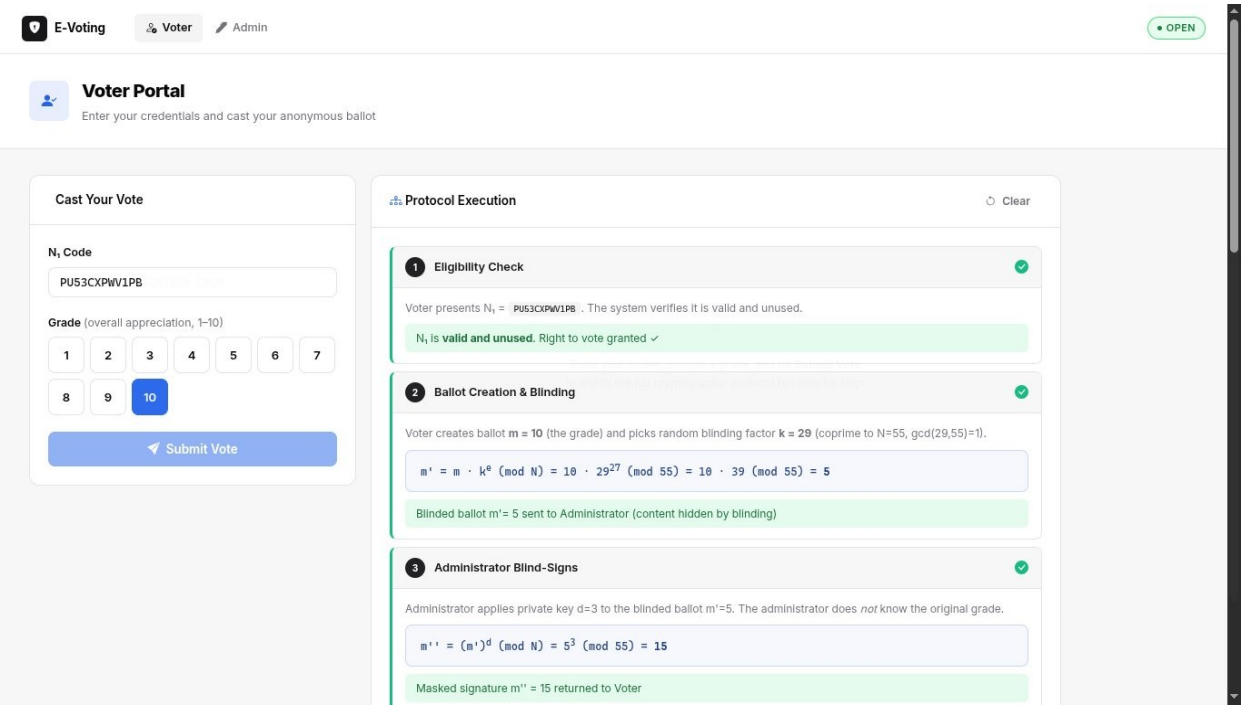

Figure 6 , Voter Portal: Live protocol execution showing blinding, blind signing, and unblinding steps

Admin Panel , Counting Results

The admin panel provides the complete election dashboard after counting. For the test election with 5 student voters:

- Total valid votes: 6 | Invalid votes: 0 | Average grade: 6.83 / 10
- Grade distribution: 2×Grade-10, 1×Grade-9, 1×Grade-6, 1×Grade-4, 1×Grade-2
- All 6 ballots: Signature valid ✓ | N2 fingerprint valid ✓ | Status: Valid

The Ballot Details table displays the N2 code for each ballot alongside its grade, signature validity status, and N2 fingerprint check result , demonstrating the complete verification chain. Any voter can look up their personal N2 code in this published table to confirm their vote was correctly recorded.

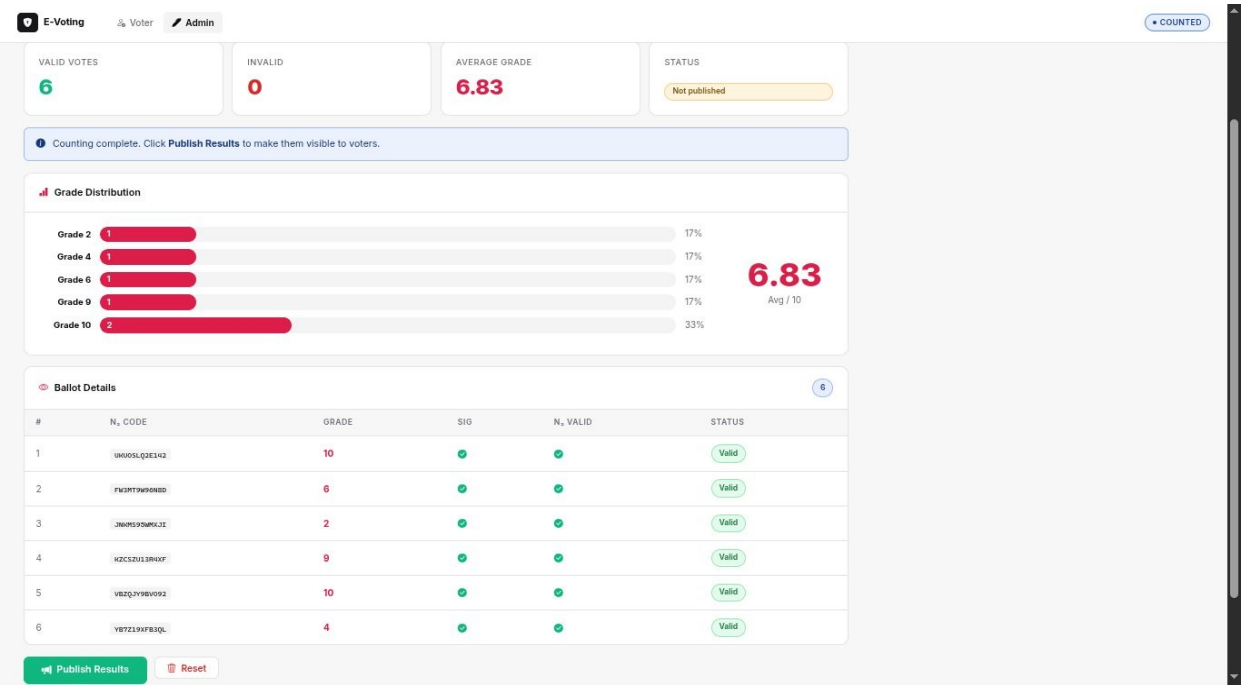

Figure 7 , Admin Panel: Counting results, grade distribution, and full ballot verification table

8.3 Key Implementation Details

RSA and Blind Signature Operations

All RSA operations use Python's built-in pow(base, exp, mod) for efficient modular exponentiation. The Cryptography library provides cryptographically secure random number generation (os.urandom) for blinding factors k. The extended Euclidean algorithm (implemented as math.gcd and a custom modular inverse function) computes d from e and φ(N) during key generation and k^{-1} during unblinding.

Ballot Serialisation and Nonce

Each ballot is serialised as a JSON object: {"vote": grade, "N2": code, "nonce": random_64bit_int}. The 64-bit nonce (random bits) ensures that two identical votes produce different plaintexts and therefore different ciphertexts, preventing any correlation attack by the anonymizer.

Election State Machine

The application enforces protocol constraints through an explicit state machine with five states: SETUP → OPEN → CLOSED → COUNTED → PUBLISHED. The administrator module refuses to sign any ballot once the state advances beyond OPEN, ensuring that the counter cannot generate new authenticated ballots to alter results post-election.

Running the Application

Prerequisites: Python 3.9+, packages: cryptography, flask. Installation and execution:

- Install dependencies: pip install cryptography flask
- Start server: python app.py
- Voter portal: http://localhost:5000/voter
- Admin panel: http://localhost:5000/administrator (password: admin123)
- Live deployment: https://cryptovote-obm3.onrender.com/ (password: admin123)

9. TEAM ORGANISATION & TASK DISTRIBUTION

The project was carried out by a group of five students. Tasks were distributed to cover the theoretical, implementation, and documentation aspects of the project:

Team Member	Responsibilities
BENZINA Mohamed	Project Lead; Comprehensive Security Analysis and Deployment; Flask-based web application architecture (app/crypto and app/routes); UI development using HTML and Flask; database management with SQLAlchemy; Project Conclusion and Insights. Exercise 3
BENCHENINA Shouhaib Meheni	RSA module (rsa_utils.py); mathematical proofs for Exercises 2; report structure and coordination; presentation (RSA Theoretical Foundation, Digital Signatures, and Hash Functions).Toy Tetragraph Hash (TTH)
DAHMOUN Mouaine Aymen	Election Preparation logic (key generation and N1/N2 voter card setup); Phase 2 Voting Process (N1 validation, blinding/unblinding protocols, and TTH hashing); Phase 3 Counting and Verification (ballot decryption, signature checks, and anonymous result publication).
MEZAACHE Abdessamie	Introduction and Core Security Requirements; Anonymizer and Counter modules; Exercise 4 completion and documentation (presentation and report).
FANTAZI Sami Ilyes	Blind Signature Scheme implementation (app/crypto/blind_signature.py); mathematical workflow for blinding, signing, and unblinding; Distributed Trust Architecture; Dual-Code Voter Scheme development (N1 eligibility and N2 anonymity); implementation of TTH hashing for ballot verification; documentation of the voter-commissioner security interface.

10. CONCLUSION

This project has demonstrated how a carefully designed combination of four cryptographic primitives , RSA encryption, digital signatures, cryptographic hash functions, and blind signatures , can produce an electronic voting protocol that is simultaneously secure, anonymous, and publicly verifiable.

The four-entity architecture (commissioner, administrator, anonymizer, counter) is the key design insight of the protocol. By distributing knowledge and trust, no single participant holds sufficient information to commit fraud, even if they were motivated to do so. Each entity's capabilities are precisely constrained by what cryptographic operations it performs and what data it is permitted to access.

The blind signature protocol , the most novel element , is elegant in its simplicity. By allowing the administrator to authenticate a ballot without seeing its content, it decouples authentication (proving the right to vote) from anonymity (hiding who voted for what). This mirrors the physical analogy of the carbon-paper envelope: the official's stamp goes through the sealed envelope without revealing the document inside.

Our Python implementation faithfully realises the complete protocol and is publicly accessible at https://cryptovote-obm3.onrender.com/. The web interface visualises each cryptographic step in real time, making it an effective educational tool for understanding applied cryptography. All test ballots were correctly encrypted, signed, transmitted, decrypted, verified, and counted, confirming the correctness of the implementation.

From a pedagogical perspective, this project has provided hands-on experience with RSA key derivation and modular arithmetic, blind signature schemes and their formal security proofs, cryptographic hash function properties and their role in protocol design, and full-stack implementation of a multi-party cryptographic protocol. These skills are directly applicable to real-world security engineering.

11. REFERENCES

[#]	Reference
[1]	Kherroubi, S. (2026). Project , Cryptography Applied to Electronic Voting. ENSTA Alger, Third Year Engineering AI & SS.
[2]	Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM, 21(2), 120–126.
[3]	Chaum, D. (1983). Blind signatures for untraceable payments. In Advances in Cryptology , CRYPTO '82, pp. 199–203. Springer, New York.

[4]	Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (2018). Handbook of Applied Cryptography. CRC Press. Available: https://cacr.uwaterloo.ca/hac/
[5]	Python Cryptography Library (cryptography.io). Documentation: https://cryptography.io/en/latest/
[6]	Flask Documentation. Pallets Projects. https://flask.palletsprojects.com/
[7]	Project Live Implementation. https://cryptovote-obm3.onrender.com/ (Admin password: admin123)

End of Report , ENSTA Alger · Academic Year 2025–2026